

向前傳播課程說明



資管研究所在人工智慧課程中關於：向前傳播 Forward Propagation
請整理出一個 mark down 的課程內容說明

以下是為「資管研究所：人工智慧課程—向前傳播（Forward Propagation）」整理的一份 Markdown 課程內容說明，語氣正式、條理清晰，適合轉成教材或簡報講義使用。

markdown

🧠 人工智慧課程：向前傳播 (Forward Propagation)

一、課程簡介

「向前傳播」(Forward Propagation) 是人工神經網路 (Artificial Neural Network, ANN) 中資料流動的第一個主要階段。

它負責將輸入資料經由各層神經元 (Neurons) 傳遞，計算出每一層的輸出，最終得到模型的****預測結果 (Predicted Output)****。

Forward Propagation 的核心概念在於：

> ****由輸入到輸出的一連串線性加權與非線性轉換過程。****

二、運作流程

1 輸入層 (Input Layer)

- 接收原始輸入資料 (如影像像素、文字特徵或數值)。
- 不進行計算，僅將資料傳入下一層。

2 隱藏層 (Hidden Layers)

每個神經元都執行以下步驟：

1. ****加權求和 (Weighted Sum)****

```
\[
z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b
\]
```

其中：

- x_i ：輸入特徵
- w_i ：權重 (Weight)
- b ：偏差 (Bias)

2. ****激勵函數 (Activation Function)****

```
\[
a = f(z)
\]
```

常見的函數有：

- Sigmoid: $f(z) = \frac{1}{1 + e^{-z}}$
- ReLU: $f(z) = \max(0, z)$
- tanh: $f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$

3 輸出層 (Output Layer)

- 根據任務型態選擇輸出形式：
 - ****迴歸問題 (Regression)****：輸出為實數值 (例如房價預測)
 - ****分類問題 (Classification)****：透過 Softmax 或 Sigmoid 輸出機率分佈

三、數學流程總覽

以兩層神經網路為例：

```
\[
\text{輸入層} \rightarrow \text{隱藏層} \rightarrow \text{輸出層}
\]
```

1. 隱藏層：

```
\[
z^{(1)} = W^{(1)}x + b^{(1)}
\]
\[
a^{(1)} = f(z^{(1)})
\]
```

2. 輸出層：

```
\[
z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}
\]
\[
\hat{y} = f(z^{(2)})
\]
```

四、流程圖示意

輸入層 → 加權求和 → 激勵函數 → 隱藏層 → 加權求和 → 激勵函數 → 輸出層

```
lua
```

(每一層的輸出，都是下一層的輸入。)

五、程式範例 (Python + NumPy)

```
```python
import numpy as np

假設有2個輸入、3個隱藏層神經元、1個輸出
X = np.array([[0.6, 0.9]])
W1 = np.array([[0.2, 0.4, 0.6],
 [0.3, 0.5, 0.7]])
b1 = np.array([[0.1, 0.1, 0.1]])
W2 = np.array([[0.8,
 [0.2,
 [0.5]])
b2 = np.array([[0.05]])

定義ReLU激勵函數
def relu(x):
 return np.maximum(0, x)

向前傳播
Z1 = np.dot(X, W1) + b1
A1 = relu(Z1)
Z2 = np.dot(A1, W2) + b2
y_hat = Z2

print("預測輸出 =", y_hat)
```

## 六、與倒傳遞（Backward Propagation）的關係

- 向前傳播：計算預測結果
- 倒傳遞：根據誤差修正權重
- 兩者構成神經網路學習的完整循環：

Forward Propagation  $\Rightarrow$  Loss Computation  $\Rightarrow$  Backward Propagation

## 七、課後思考題

1. 為什麼激勵函數在神經網路中不可或缺？
2. 若隱藏層全為線性函數，模型會有何限制？
3. Forward Propagation 與 Backward Propagation 在資料流向上有何差異？

## 八、延伸閱讀

- Goodfellow, Bengio & Courville (2016). *Deep Learning*. MIT Press.
- TensorFlow 官方文件：<https://www.tensorflow.org/guide/keras>
- PyTorch 教學範例：<https://pytorch.org/tutorials>

yaml

---

是否希望我幫你把這份 Markdown \*\*轉成 PowerPoint (.pptx)\*\* 課程簡報（含圖表與流程圖）？  
我可以直接生成可下載的 .pptx 檔案。



上一份為何有\及frac, 似乎是符號, 太亂了  
麻煩再重新產生一份新的, 課程內容

非常好 🍌

以下是重新整理後的版本 —

我去掉了所有 LaTeX 符號（如  $\frac{1}{2}$  等），改用「易讀文字+簡單公式格式」，讓整份 Markdown 更清楚、乾淨，適合放在教學講義或投影片中。

markdown

# 🧠 人工智慧課程：向前傳播（Forward Propagation）

## 一、課程簡介

向前傳播（Forward Propagation）是人工神經網路中資料「由輸入到輸出」的傳遞過程。  
主要任務是：根據每層神經元的權重（Weight）與偏差（Bias），計算出網路的預測值。

簡單來說，就是：

> 將輸入資料逐層傳遞、加權、並經過激勵函數轉換，直到產生最終輸出。

---

## ## 二、運作流程

### ### 1 輸入層 (Input Layer)

- 接收原始資料，例如：
  - 影像像素值
  - 數值型特徵
  - 文字轉換後的向量
- 不做運算，只負責把資料送入下一層。

---

### ### 2 隱藏層 (Hidden Layer)

每一個神經元都會執行以下兩個步驟：

**\*\* (1) 加權求和：\*\***

$$z = w_1 \times x_1 + w_2 \times x_2 + \dots + w_n \times x_n + b$$

**\*\* (2) 激勵函數：\*\***

$$a = f(z)$$

常見的激勵函數：

- Sigmoid:  $f(z) = 1 / (1 + e^{(-z)})$
- ReLU:  $f(z) = \max(0, z)$
- tanh:  $f(z) = (e^z - e^{(-z)}) / (e^z + e^{(-z)})$

---

### ### 3 輸出層 (Output Layer)

根據任務類型，輸出層的形式不同：

類型	輸出形式	範例
迴歸問題	連續數值	房價預測、氣溫預測
分類問題	機率分佈	圖像辨識、文字分類

---

## ## 三、整體流程圖（概念示意）

輸入層 → 加權求和 → 激勵函數 → 隱藏層 → 加權求和 → 激勵函數 → 輸出層

lua

每一層的輸出，都是下一層的輸入。

---

## ## 四、簡單的計算範例

假設有兩個輸入  $x_1=0.6$ 、 $x_2=0.9$

權重  $w_1=0.2$ 、 $w_2=0.3$ ，偏差  $b=0.1$

則：

1. 計算加權和：

$$z = (0.2 * 0.6) + (0.3 * 0.9) + 0.1 = 0.53$$

2. 使用 ReLU 激勵函數：

```
a = max(0, 0.53) = 0.53
```

3. 此輸出 a 會傳入下一層繼續運算。

```

```

## 五、Python 程式範例

```
```python
import numpy as np

# 假設有2個輸入、3個隱藏層神經元、1個輸出
X = np.array([[0.6, 0.9]])
W1 = np.array([[0.2, 0.4, 0.6],
               [0.3, 0.5, 0.7]])
b1 = np.array([0.1, 0.1, 0.1])
W2 = np.array([0.8,
               0.2,
               0.5])
b2 = np.array([0.05])

def relu(x):
    return np.maximum(0, x)

# 向前傳播
Z1 = np.dot(X, W1) + b1
A1 = relu(Z1)
Z2 = np.dot(A1, W2) + b2
Y_pred = Z2

print("預測輸出 =", Y_pred)
```

六、與倒傳遞（Backward Propagation）的關係

階段	目的	資料流向
向前傳播	計算預測結果	由輸入 → 輸出
倒傳遞	修正權重與偏差	由輸出 → 輸入（反向）

兩者構成神經網路訓練的核心循環：

```
mathematica
```

Forward Propagation → 損失計算 → Backward Propagation → 權重更新

七、課後思考

1. 為什麼神經網路需要激勵函數？
2. 若隱藏層都不使用非線性函數，模型能力會受到什麼限制？
3. 向前傳播與倒傳遞在資料流方向上有何不同？

八、延伸閱讀

- Goodfellow, Bengio & Courville, 《Deep Learning》，MIT Press
- TensorFlow 官方文件：<https://www.tensorflow.org/guide/keras> ↗

- PyTorch 教學範例：<https://pytorch.org/tutorials>[↗]

```
yaml
```

```
---
```

是否要我幫這份 Markdown 直接轉成 PowerPoint 簡報（含流程圖與頁面設計）？
我可以幫你自動生成 ``.pptx`` 檔案，使用繁體中文介面與教學風格版面。